

# BINARY SEARCH TREE(BST)

## Introduction to BST

- A Binary Search Tree (BST) is a fundamental data structure used for organizing and storing elements in a way that facilitates efficient searching, insertion, and deletion operations.
- It is a type of binary tree, meaning that each node in the tree can have at most two children, a left child and a right child.

The key characteristic of a BST is the arrangement of elements within the tree, which adheres to the following rules:

### Binary Tree Structure:

- ✓ Each node has at most two children, a left child and a right child.
- ✓ The left subtree of a node contains only elements that are less than the node's value.
- ✓ The right subtree of a node contains only elements that are greater than the node's value.

### Ordering Property:

- ✓ The values stored in the BST are organized in a way that ensures a specific order.
- ✓ For any given node, all elements in its left subtree are smaller than the node, and all elements in its right subtree are larger than the node.

### No Duplicates:

- ✓ In typical BST implementations, each element must be unique. Duplicate values are not allowed.

- The **binary search** property enables efficient search operations, making the average **time complexity for search, insertion, and deletion operations  $O(\log n)$** , where  $n$  is the number of nodes in the tree.
- However, the performance can degrade to  **$O(n)$**  in the worst case if the tree becomes unbalanced.

## Basic Operations on a BST:

### Search:

- ✓ To find a specific element in the BST, you start at the root and recursively navigate the tree by comparing the target value with the values of nodes.

### Insertion:

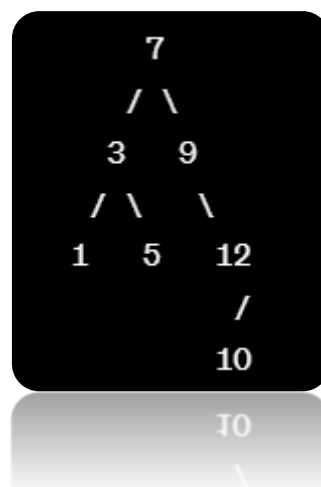
- ✓ To insert a new element into the BST, you traverse the tree based on the binary search property to find the appropriate position.
- ✓ Once found, a new node is inserted.

### Deletion:

- ✓ Deleting a node involves finding the node to be deleted and rearranging the tree to maintain the binary search property.

### Example:

Consider the following **BST**:



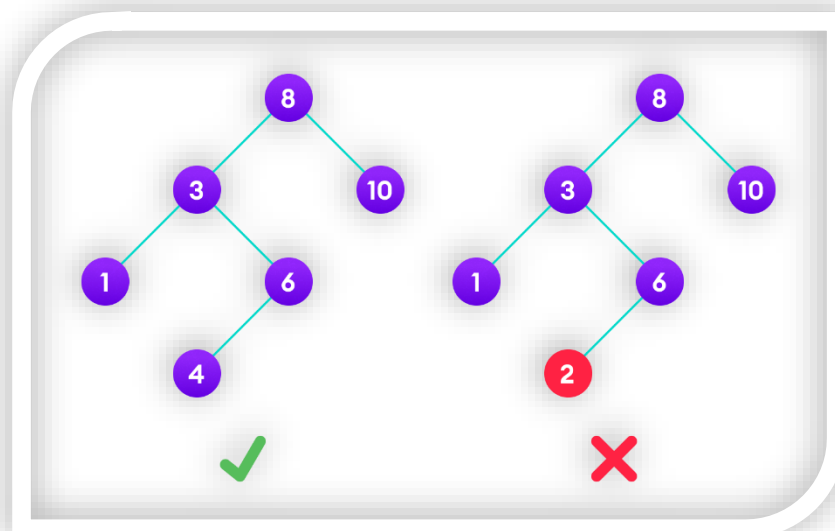
👉 In this example, the binary search property is maintained: all nodes in the **left subtree** of a node have values **less than** the node, and all nodes in the **right subtree** have values **greater than** the node.

- **Binary Search Trees** are a foundational concept in computer science and are widely used in various applications, including databases, compilers, and search algorithms.
- However, ensuring balance in a BST is crucial for maintaining optimal performance, leading to the development of self-balancing variants like **AVL trees** and **Red-Black trees**.

## Properties of BST

The properties that separate a binary search tree from a regular binary tree is:

- All nodes of left subtree are less than the root node
- All nodes of right subtree are more than the root node
- Both subtrees of each node are also BSTs i.e. they have the above two properties



A tree having a **right subtree** with one value smaller than the root is shown to demonstrate that it is not a **valid binary** search tree.

## Search Operation

- The algorithm depends on the property of BST that if each left subtree has values below root and each right subtree has values above the root.
- If the value is below the root, we can say for sure that the value is not in the right subtree; we need to only search in the left subtree and if the value is above the root, we can say for sure that the value is not in the left subtree; we need to only search in the right subtree.

### Algorithm:

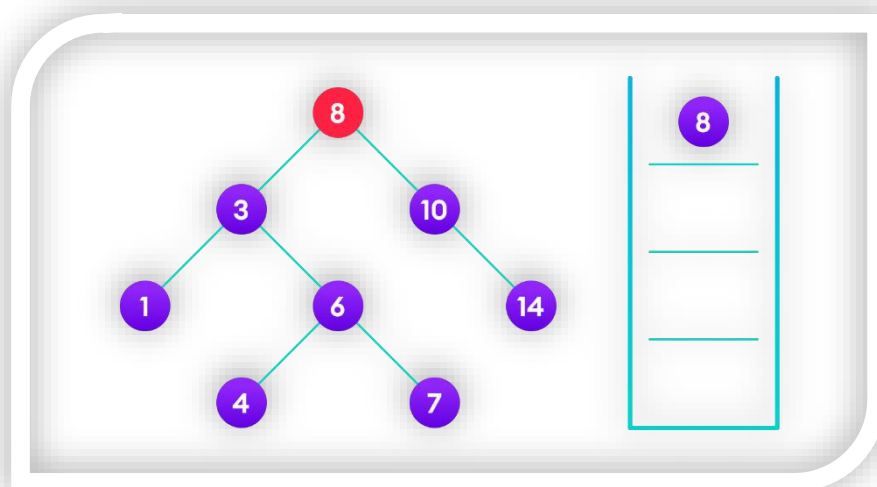
```
If root == NULL
return NULL;

If number == root->data
return root->data;

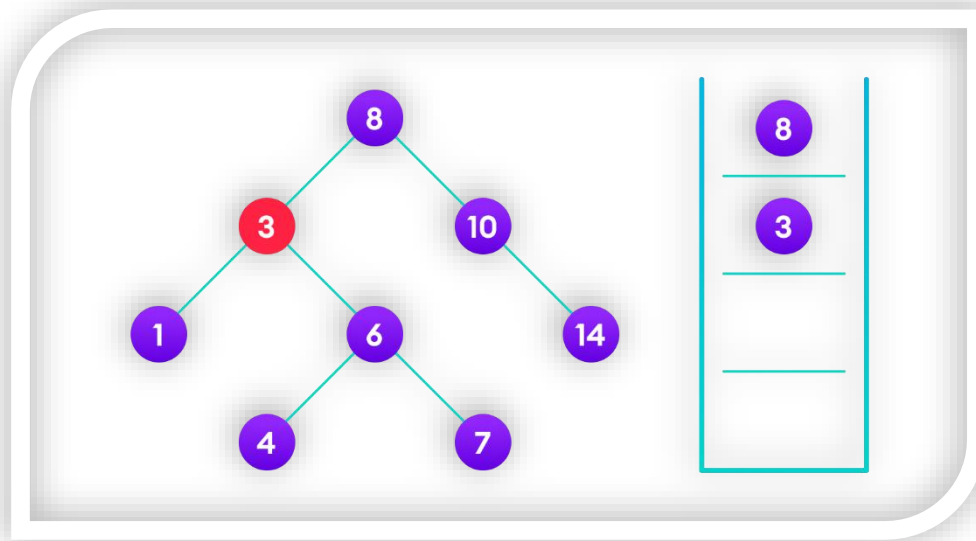
If number < root->data
return search(root->left)

If number > root->data
return search(root->right)
```

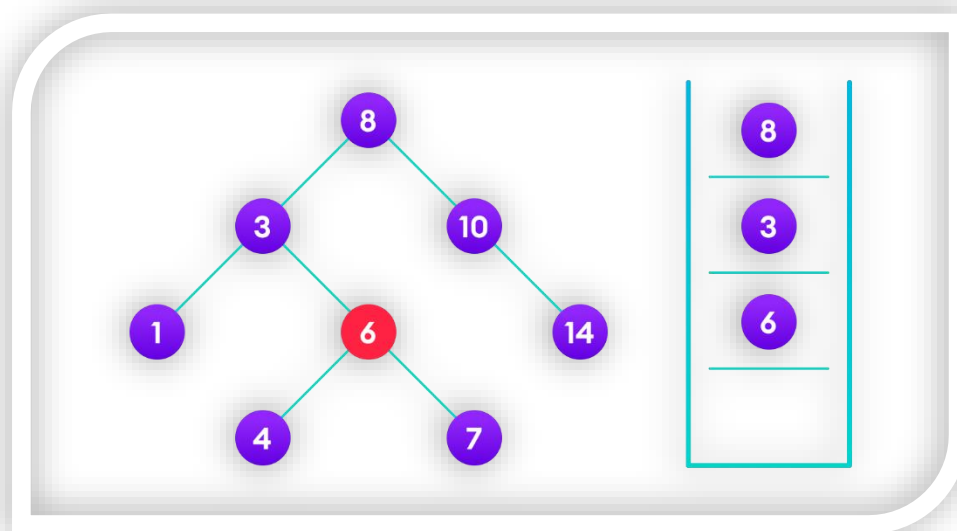
Let us try to visualize this with a diagram.



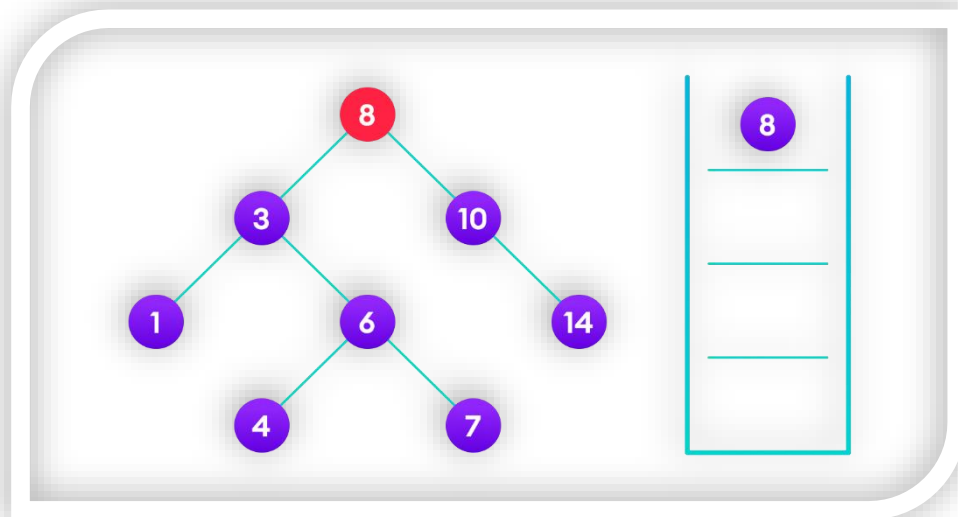
4 is not found so, traverse through the left subtree of 8



4 is not found so, traverse through the right subtree of 3

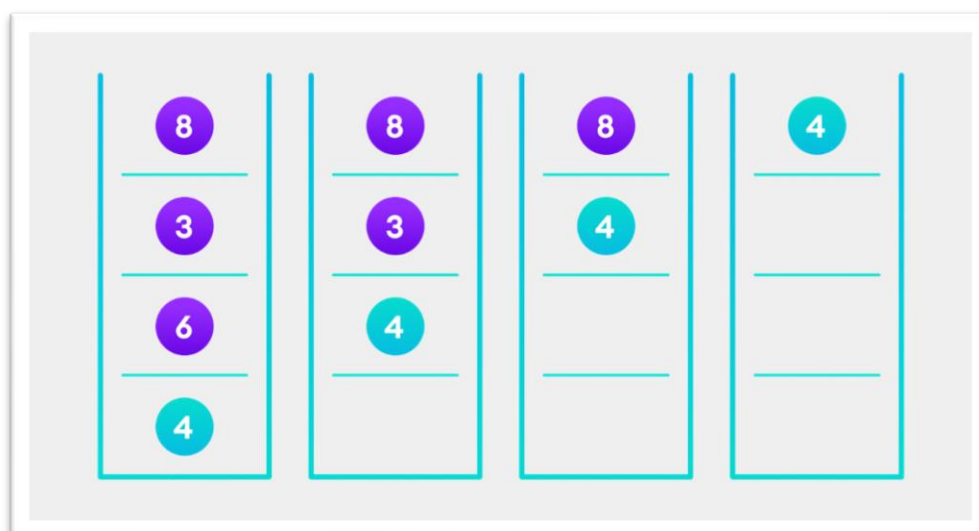


4 is not so, traverse through the left subtree of 6



4 is found

- ✓ If the value is found, we return the value so that it gets propagated in each recursion step as shown in the image below.
- ✓ If you might have noticed, we have called `return search(struct node*)` four times. When we return either the new node or NULL, the value gets returned again and again until `search(root)` returns the final result.



If the value is not found, we eventually reach the left or right child of a leaf node which is NULL and it gets propagated and returned

## Implementation of Searching Tree in BST:

```
#include <stdio.h>

#include <stdlib.h>

// Definition of a tree node
struct TreeNode {
    int key;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Function to create a new tree node
struct TreeNode* createNode(int key) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->key = key;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to search for a key in the BST
struct TreeNode* searchBST(struct TreeNode* root, int key) {
    // Base case: if the tree is empty or the key is found
    if (root == NULL || root->key == key) {
        return root;
    }

    // If the key is smaller than the root's key, search in the left subtree
    if (key < root->key) {
        return searchBST(root->left, key);
    }

    // If the key is larger than the root's key, search in the right subtree
    return searchBST(root->right, key);
}
```

```
// Example usage:

int main() {

    // Constructing a BST

    //      5
    //     /\
    //    3  7
    //   /\ /\
    //  2 4 6 8

    struct TreeNode* root = createNode(5);

    root->left = createNode(3);
    root->right = createNode(7);
    root->left->left = createNode(2);
    root->left->right = createNode(4);
    root->right->left = createNode(6);
    root->right->right = createNode(8);

    // Searching for a key in the BST

    int keyToSearch = 4;

    struct TreeNode* resultNode = searchBST(root, keyToSearch);

    if (resultNode != NULL) {
        printf("Key %d found in the BST.\n", keyToSearch);
    } else {
        printf("Key %d not found in the BST.\n", keyToSearch);
    }

    // Don't forget to free the allocated memory to avoid memory leaks
    // (This is a simplified example and may not handle all edge cases)

    free(root->left->left);
    free(root->left->right);
    free(root->left);
    free(root->right->left);
    free(root->right->right);
    free(root->right);
    free(root);

    return 0; }

```



OUTPUT:

Key 4 found in the BST.

- ✓ Search operation is  $O(\log n)$ .
- ✓ The time complexity could approach  $O(n)$

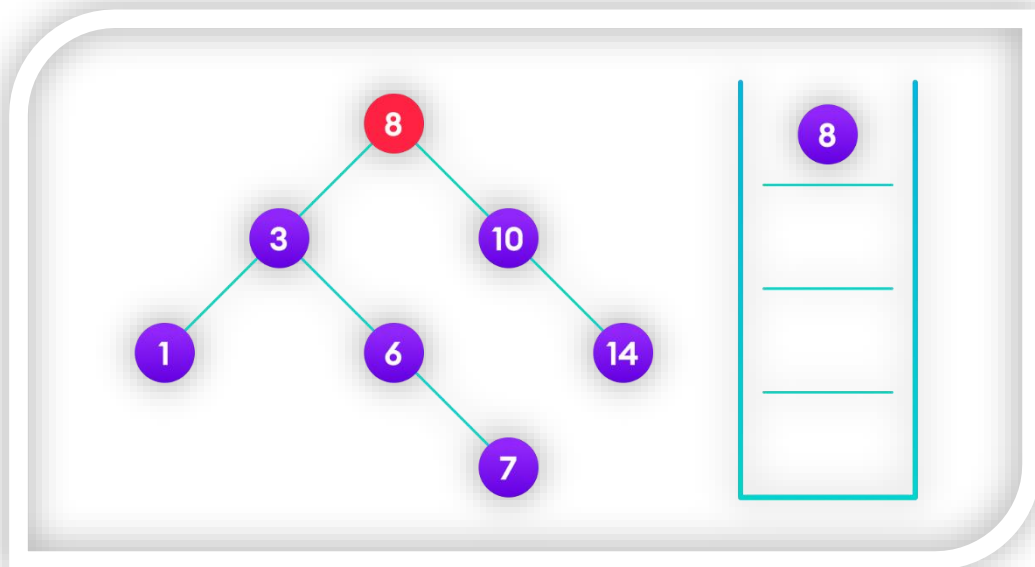
## Insert Operation

- **Inserting** a value in the correct position is similar to searching because we try to maintain the rule that the left subtree is lesser than root and the right subtree is larger than root.
- We keep going to either right subtree or left subtree depending on the value and when we reach a point left or right subtree is null, we put the new node there.

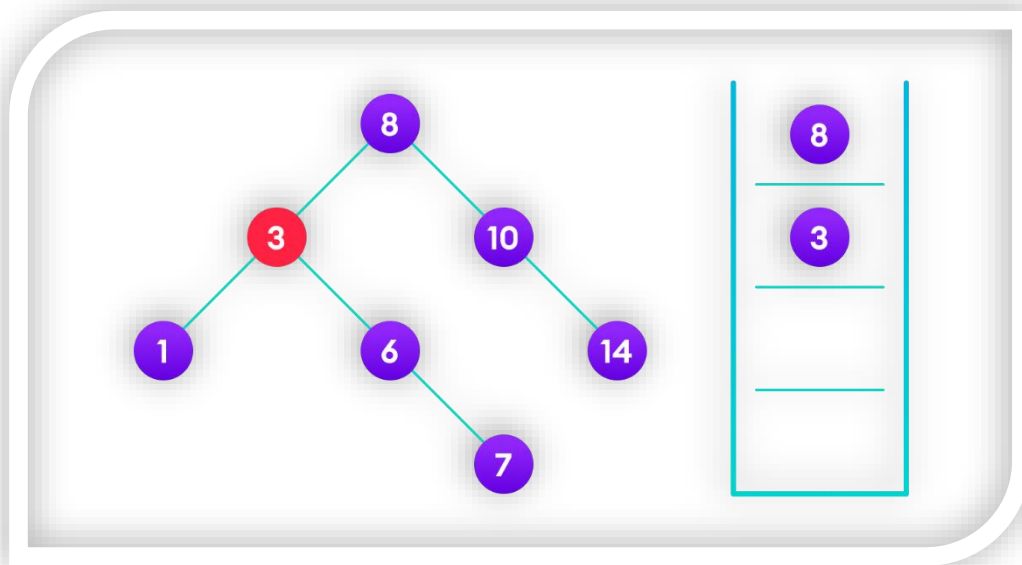
**Algorithm:**

```
1. If node == NULL
2. return createNode(data)
3. if (data < node->data)
4. node->left = insert(node->left, data);
5. else if (data > node->data)
6. node->right = insert(node->right, data);
7. return node;
```

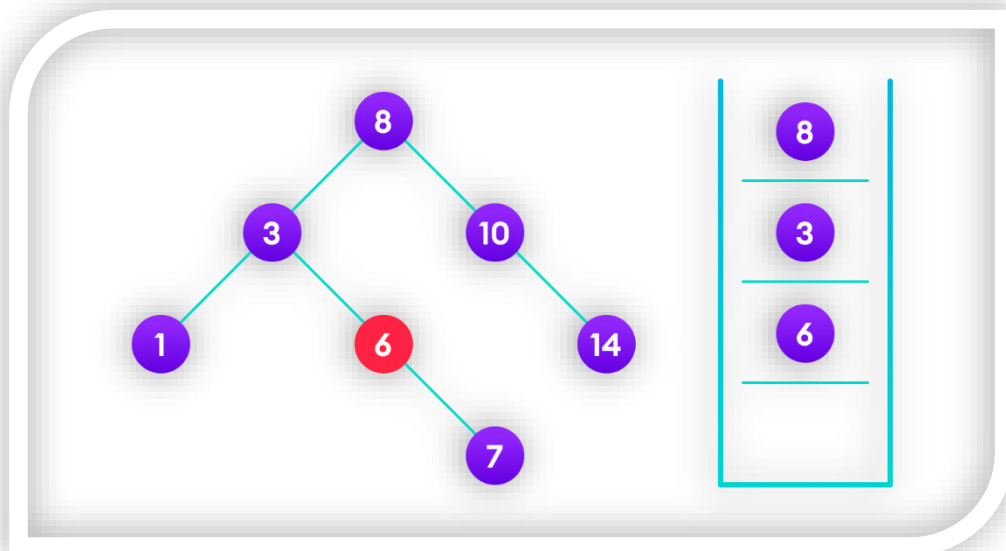
The algorithm isn't as simple as it looks. Let's try to visualize how we add a number to an existing BST.



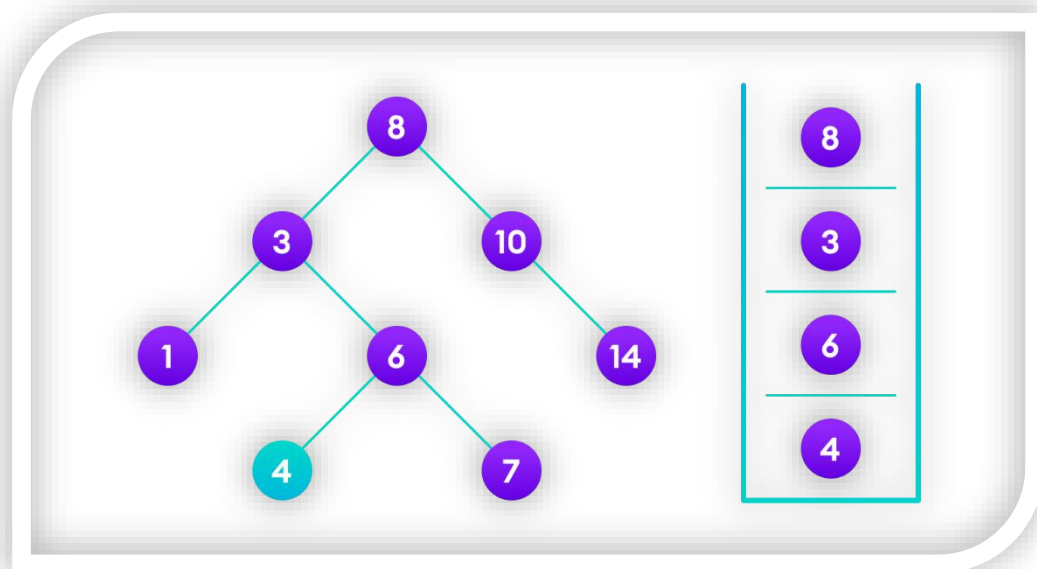
$4 < 8$  so, transverse through the left child of 8



$4 > 3$  so, transverse through the right child of 8



$4 < 6$  so, transverse through the left child of 6



Insert 4 as a left child of 6

- ✓ We have attached the node but we still have to exit from the function without doing any damage to the rest of the tree.
- ✓ This is where the **return node**; at the end comes in handy.
- ✓ In the case of **NULL**, the newly created node is returned and attached to the parent node, otherwise the same node is returned without any change as we go up until we return to the root.

- ✓ This makes sure that as we move back up the tree, the other node connections aren't changed.

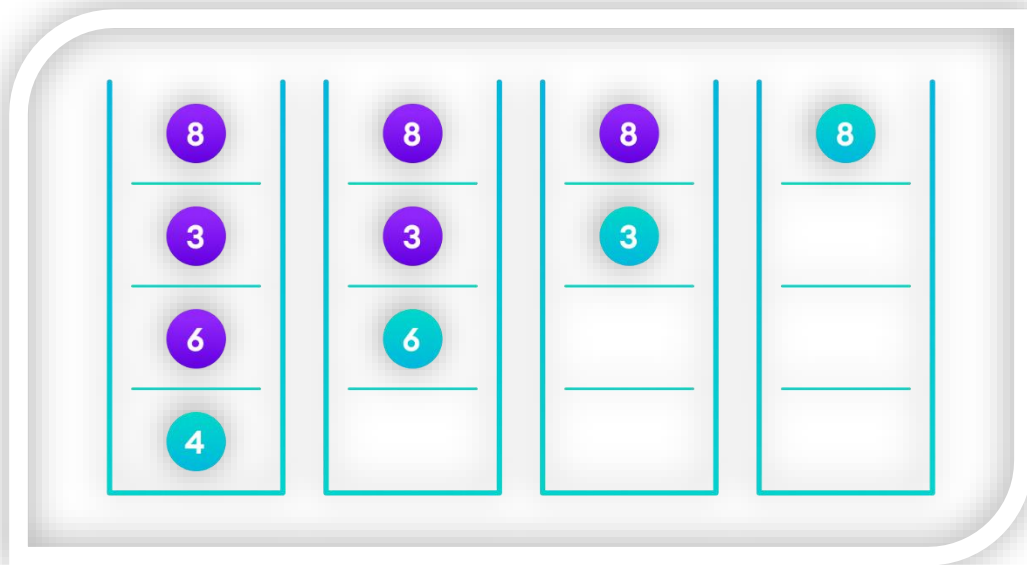


Image showing the importance of returning the root element at the end so that the elements don't lose their position during the upward recursion step.

### Implementation of Insertion:

```
#include <stdio.h>
#include <stdlib.h>
// Definition of a tree node
struct TreeNode {
    int key;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

```
// Function to create a new tree node
struct TreeNode* createNode(int key) {
    struct TreeNode* newNode = (struct
    TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->key = key;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to insert a key into the BST
struct TreeNode* insertBST(struct TreeNode* root,
int key) {
    // Base case: if the tree is empty, create a new node
    if (root == NULL) {
        return createNode(key);
    }

    // Recursive cases: insert into the left or right
    subtree
    if (key < root->key) {
        root->left = insertBST(root->left, key);
    } else if (key > root->key) {
        root->right = insertBST(root->right, key);
    }

    // Return the modified tree
    return root;
}
```

```
// In-order traversal to print the BST
void inOrderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->key);
        inOrderTraversal(root->right);
    }
}

// Example usage:
int main() {
    // Constructing an empty BST
    struct TreeNode* root = NULL;

    // Inserting keys into the BST
    root = insertBST(root, 5);
    root = insertBST(root, 3);
    root = insertBST(root, 7);
    root = insertBST(root, 2);
    root = insertBST(root, 4);
    root = insertBST(root, 6);
    root = insertBST(root, 8);

    // Printing the in-order traversal of the BST
    printf("In-order traversal of the BST: ");
    inOrderTraversal(root);

    printf("\n");

    // Don't forget to free the allocated memory to avoid
    memory leaks

    // (This is a simplified example and may not handle all edge
    cases)

    // Note: Normally, freeing memory would be done when the
    BST is no longer needed.

    return 0;
}
```

**Time Complexity:  $O(\log n)$**

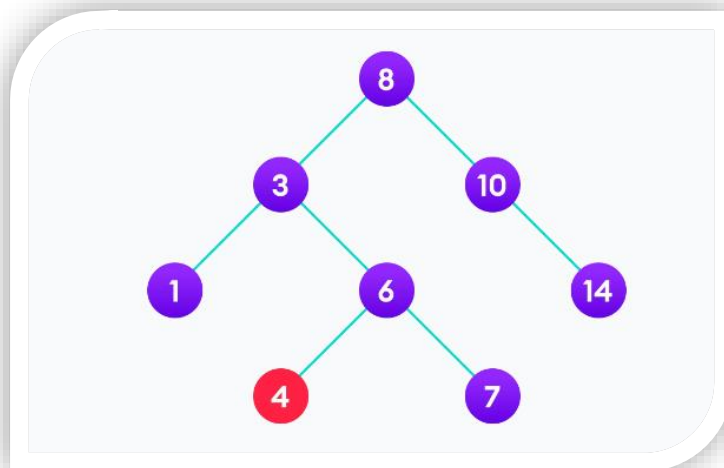
**Space Complexity:  $O(\log n)$**

## Deletion Operation

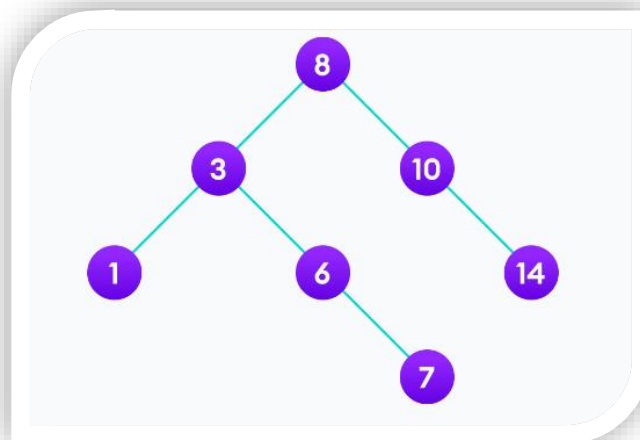
There are three cases for deleting a node from a binary search tree.

### Case I

- ✓ In the first case, the node to be deleted is the leaf node. In such a case, simply delete the node from the tree.



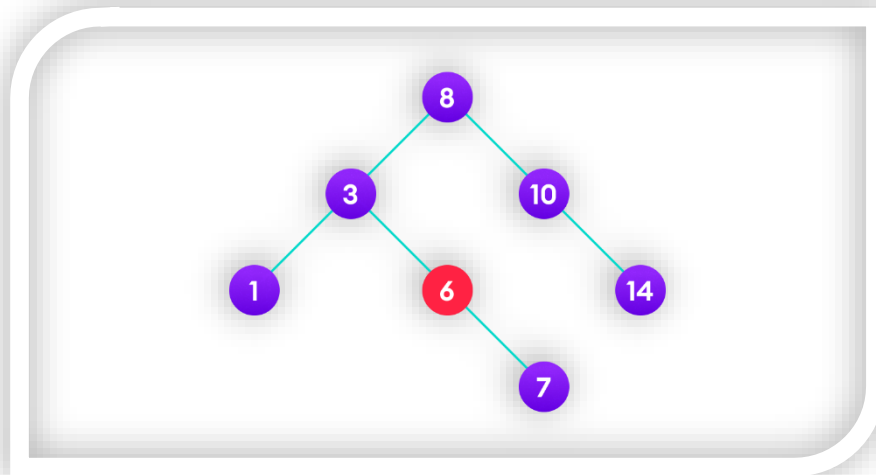
4 is to be deleted



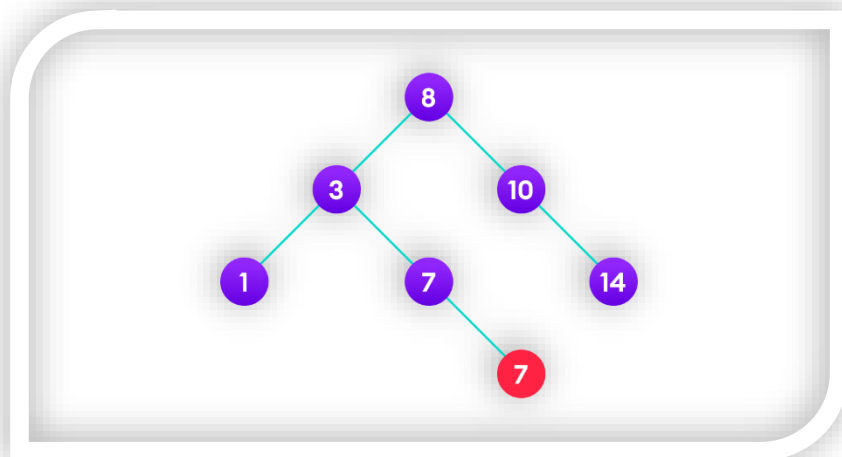
Delete the node

**Case II**

- ✓ In the second case, the node to be deleted lies has a single child node.  
In such a case follow the steps below:
  1. Replace that node with its child node.
  2. Remove the child node from its original position.

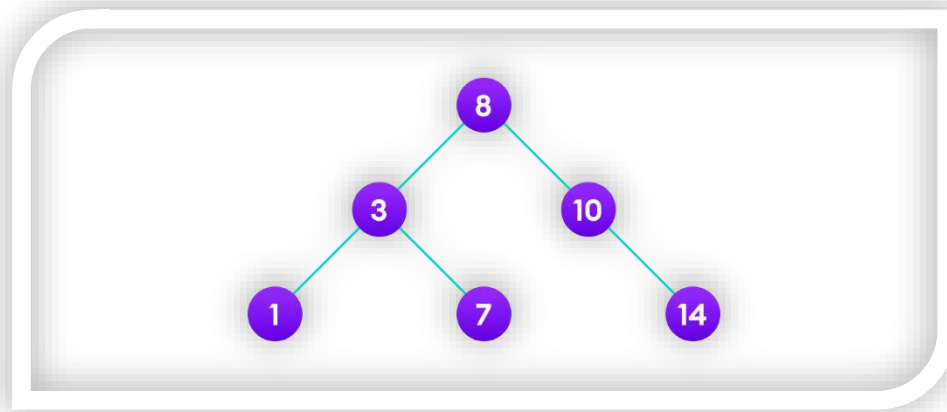


6 is to be deleted



copy the value of its child to the node and delete the child



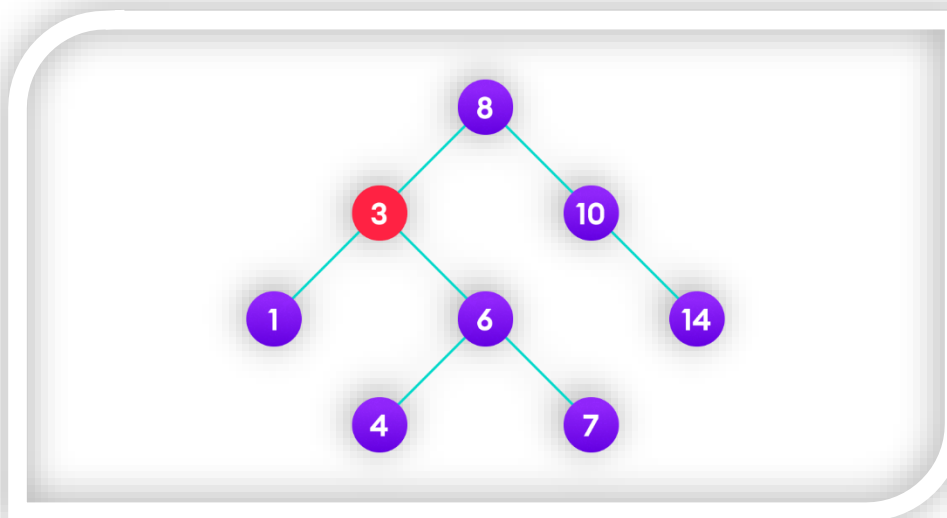


Final node

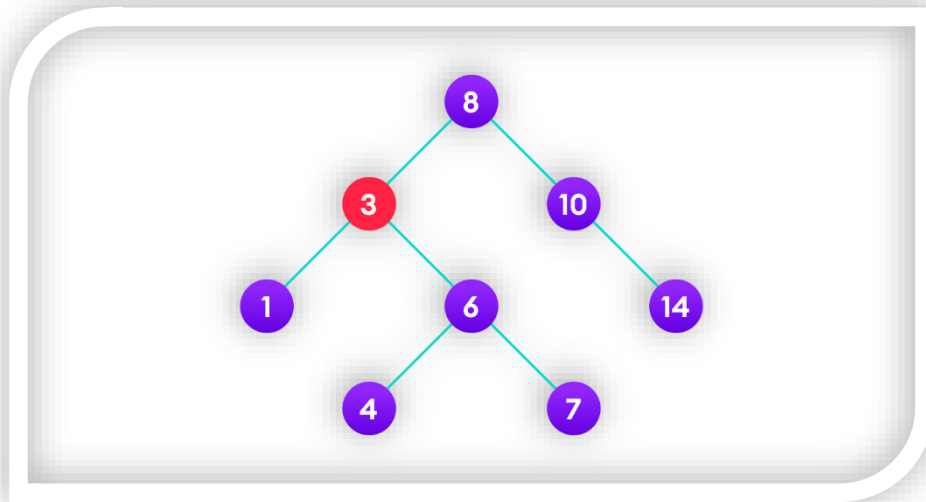
**Case III**

In the third case, the node to be deleted has two children. In such a case follow the steps below:

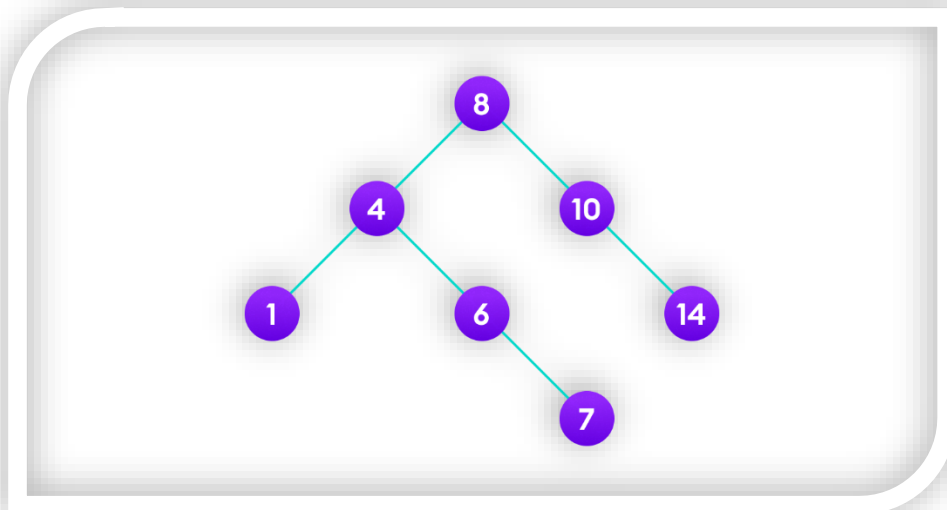
1. Get the inorder successor of that node.
2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.



3 is to be deleted



Copy the value of the inorder successor (4) to the node.



Delete the inorder successor

## Implementation of Deletion in BST:

```
#include <stdio.h>
#include <stdlib.h>

// Definition of a tree node
struct TreeNode {
    int key;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Function to create a new tree node
struct TreeNode* createNode(int key) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct
TreeNode));

    newNode->key = key;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to find the node with the minimum key in a BST
struct TreeNode* findMinNode(struct TreeNode* node) {
    while (node->left != NULL) {
        node = node->left;
    }
    return node;
}
```

```

// Function to delete a key in the BST
struct TreeNode* deleteBST(struct TreeNode* root, int key) {
    // Base case: if the tree is empty
    if (root == NULL) {
        return root;
    }
    // Recursive cases: find the node to be deleted
    if (key < root->key) {
        root->left = deleteBST(root->left, key);
    } else if (key > root->key) {
        root->right = deleteBST(root->right, key);
    } else {
        // Node with only one child or no child
        if (root->left == NULL) {
            struct TreeNode* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct TreeNode* temp = root->left;
            free(root);
            return temp;
        }
        // Node with two children: get the inorder successor (smallest
        // in the right subtree) and replace the current node's key
        struct TreeNode* temp = findMinNode(root->right);
        root->key = temp->key;
        root->right = deleteBST(root->right, temp->key);
    }
    return root;
}

```

```
// Example usage:
int main() {
    // Constructing a BST
    struct TreeNode* root = createNode(5);
    root->left = createNode(3);
    root->right = createNode(7);
    root->left->left = createNode(2);
    root->left->right = createNode(4);
    root->right->left = createNode(6);
    root->right->right = createNode(8);

    // Printing the in-order traversal of the original BST
    printf("In-order traversal before deletion: ");
    inOrderTraversal(root);
    printf("\n");

    // Deleting a key from the BST
    int keyToDelete = 5;
    root = deleteBST(root, keyToDelete);

    // Printing the in-order traversal after deletion
    printf("In-order traversal after deletion of key %d: ", keyToDelete);
    inOrderTraversal(root);
    printf("\n");
}
```

```
// In-order traversal to print the BST
void inOrderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->key);
        inOrderTraversal(root->right);
    }
}

// Don't forget to free the allocated memory to avoid memory leaks
// (This is a simplified example and may not handle all edge cases)
free(root->left->left);
free(root->left->right);
free(root->left);
free(root->right->left);
free(root->right->right);
free(root->right);
free(root);
return 0;
}
```

### **Real-time Application of Binary Search tree:**

- ✓ BSTs are used for indexing in databases.
- ✓ It is used to implement searching algorithms.
- ✓ BSTs are used to implement Huffman coding algorithm.
- ✓ It is also used to implement dictionaries.
- ✓ Used for data caching.
- ✓ Used in Priority queues.
- ✓ Used in spell checkers.

### **Advantages of Binary Search Tree:**

- ✓ BST is fast in insertion and deletion when balanced. It is fast with a time complexity of  $O(\log n)$ .
- ✓ BST is also for fast searching, with a time complexity of  $O(\log n)$  for most operations.
- ✓ BST is efficient. It is efficient because they only store the elements and do not require additional memory for pointers or other data structures.
- ✓ We can also do range queries – find keys between N and M ( $N \leq M$ ).
- ✓ BST code is simple as compared to other data structures.
- ✓ BST can automatically sort elements as they are inserted, so the elements are always stored in a sorted order.
- ✓ BST can be easily modified to store additional data or to support other operations. This makes it flexible.

### **Disadvantages of Binary Search Tree:**

- ✓ The main disadvantage is that we should always implement a balanced binary search tree. Otherwise the cost of operations may not be logarithmic and degenerate into a linear search on an array.
- ✓ They are not well-suited for data structures that need to be accessed randomly, since the time complexity for search, insert, and delete operations is  $O(\log n)$ , which is good for large data sets, but not as fast as some other data structures such as arrays or hash tables.